

CRUD Lab (Practical - Full CRUD Implementation & Modular Routing)

Welcome to the practical lab for CRUD! In this session, you will restructure your backend using `APIRouter` and implement full **CRUD (Create, Read, Update, Delete)** operations to manage your machine learning prediction logs in MySQL.

1. Concept Overview

In this lab, you will:

- Refactor your codebase from a single file into a modular structure using FastAPI's `APIRouter`.
- Implement all four CRUD database endpoints:
- **Create (POST)**: Save a new feature submission and its prediction result.
- **Read (GET)**: Fetch all prediction logs or a specific record by its ID.
- **Update (PUT)**: Modify an existing record's features or results.
- **Delete (DELETE)**: Remove an outdated or incorrect record from MySQL.

2. Step-by-Step Lab Instructions

Step 1: Create a Modular Directory Structure

Organize your project folder in Visual Studio Code with the following structure:

```
ml_backend/
├── database.py          # SQLAlchemy engine and session dependency
├── models.py           # SQLAlchemy database ORM models
├── schemas.py          # Pydantic validation schemas (Input & Output)
├── routers/
│   └── predictions.py  # APIRouter containing all CRUD endpoints
└── main.py             # FastAPI app instance including routers
```

Step 2: Define Pydantic Schemas (`schemas.py`)

Create `schemas.py` to handle data validation for creation, updating, and response formatting:

```
# schemas.py
from pydantic import BaseModel
import datetime

# Schema for incoming prediction creation request
class PredictionCreate(BaseModel):
    feature1: float
    feature2: float

# Schema for updating an existing prediction record
class PredictionUpdate(BaseModel):
    feature1: float | None = None
    feature2: float | None = None
    prediction_result: float | None = None

# Schema for returning records to the client
class PredictionResponse(BaseModel):
    id: int
    feature1: float
    feature2: float
    prediction_result: float
    created_at: datetime.datetime

class Config:
    from_attributes = True
```

Step 3: Implement CRUD Router (`routers/predictions.py`)

Create `routers/predictions.py` to house all database operations:

```
# routers/predictions.py
from fastapi import APIRouter, Depends, HTTPException, status
from sqlalchemy.orm import Session
from database import get_db
from models import PredictionRecord
from schemas import PredictionCreate, PredictionUpdate, PredictionResponse
from typing import List

router = APIRouter(prefix="/predictions", tags=["Predictions CRUD"])

# 1. CREATE: Save a new prediction record
@router.post("/", response_model=PredictionResponse,
status_code=status.HTTP_201_CREATED)
def create_prediction(payload: PredictionCreate, db: Session =
Depends(get_db)):
    # Calculate mock ML prediction score
    score = (payload.feature1 + payload.feature2) * 1.5
```

```

        db_record = PredictionRecord(
            feature1=payload.feature1,
            feature2=payload.feature2,
            prediction_result=round(score, 2)
        )
        db.add(db_record)
        db.commit()
        db.refresh(db_record)
        return db_record

# 2. READ: Get all prediction logs
@router.get("/", response_model=List[PredictionResponse])
def get_all_predictions(skip: int = 0, limit: int = 100, db: Session = Depends(get_db)):
    records = db.query(PredictionRecord).offset(skip).limit(limit).all()
    return records

# 3. READ: Get a single prediction record by ID
@router.get("/{record_id}", response_model=PredictionResponse)
def get_prediction_by_id(record_id: int, db: Session = Depends(get_db)):
    record = db.query(PredictionRecord).filter(PredictionRecord.id == record_id).first()
    if not record:
        raise HTTPException(status_code=404, detail="Prediction record not found")
    return record

# 4. UPDATE: Modify an existing prediction record
@router.put("/{record_id}", response_model=PredictionResponse)
def update_prediction(record_id: int, payload: PredictionUpdate, db: Session = Depends(get_db)):
    record = db.query(PredictionRecord).filter(PredictionRecord.id == record_id).first()
    if not record:
        raise HTTPException(status_code=404, detail="Prediction record not found")

    # Update fields if provided
    if payload.feature1 is not None:
        record.feature1 = payload.feature1
    if payload.feature2 is not None:
        record.feature2 = payload.feature2
    if payload.prediction_result is not None:
        record.prediction_result = payload.prediction_result

    db.commit()
    db.refresh(record)
    return record

# 5. DELETE: Remove a prediction record
@router.delete("/{record_id}", status_code=status.HTTP_204_NO_CONTENT)
def delete_prediction(record_id: int, db: Session = Depends(get_db)):

```

```

        record = db.query(PredictionRecord).filter(PredictionRecord.id ==
record_id).first()
        if not record:
            raise HTTPException(status_code=404, detail="Prediction record not
found")

        db.delete(record)
        db.commit()
        return None

```

Step 4: Mount Router in `main.py`

Update your root `main.py` file to include the new prediction router:

```

# main.py
from fastapi import FastAPI
from database import engine, Base
from routers import predictions

# Create database tables automatically
Base.metadata.create_all(bind=engine)

app = FastAPI(title="Modular ML CRUD Backend", version="1.0")

# Include modular router
app.include_router(predictions.router)

@app.get("/")
def read_root():
    return {"message": "Modular FastAPI backend with full MySQL CRUD
active!"}

```

3. Testing Your CRUD Endpoints

1. Start your server:

```
uvicorn main:app --reload
```

2. Open your browser to `[http://127.0.0.1:8000/docs]` (`http://127.0.0.1:8000/docs`) to view Swagger UI. You will now see all your organized CRUD endpoints categorized under **Predictions CRUD**:

- `POST /predictions/` : Submit features to create a new log.
- `GET /predictions/` : Retrieve all saved prediction logs.
- `GET /predictions/{id}` : Fetch a specific log by its ID.
- `PUT /predictions/{id}` : Update an existing record.
- `DELETE /predictions/{id}` : Delete a record from MySQL.

4. Reference Links & Further Reading

- **FastAPI Bigger Applications Guide:** Deep dive into organizing projects with multiple routers and directories.
 - **SQLAlchemy Querying Data:** Official documentation on filtering, updating, and deleting ORM records.
-